Руководство разработчика Rockchip Linux Rockit

Идентификатор документа: RK-KF-YF-532

Версия выпуска: V0.8.0

Дата: 2020-10-28

Уровень конфиденциальности: совершенно секретно секретно внутренний документ открытый документ

Отказ от ответственности

Настоящий документ предоставляется «как есть». Rockchip Electronics Co., Ltd. («Компания», далее по тексту) не предоставляет никаких явных или подразумеваемых заявлений или гарантий в отношении любых утверждений, сведений и материалов настоящего документа, включая их точность, надёжность, полноту, пригодность для продажи, пригодность для определённой цели и отсутствие нарушения прав. Настоящий документ предназначен только для использования в качестве справочного руководства.

В связи с обновлением версии продукта или по другим причинам настоящий документ может периодически обновляться или изменяться без предварительного уведомления.

Заявление о товарных знаках

«Rockchip» и соответствующие наименования являются зарегистрированными товарными знаками Компании и принадлежат Компании.

Все прочие зарегистрированные товарные знаки и товарные знаки, которые могут упоминаться в настоящем документе, принадлежат их соответствующим владельцам.

© 2020 Rockchip Electronics Co., Ltd. Все права защищены.

За пределами разумного использования без письменного разрешения Компании ни одна организация и ни одно физическое лицо не вправе самовольно извлекать или копировать часть либо весь текст настоящего документа, а также распространять его в какой-либо форме.

Rockchip Electronics Co., Ltd.

Rockchip Electronics Co., Ltd.

Адрес: здание 18, зона A, Software Park, Tongpan Road, г. Фучжоу, провинция Фуцзянь

Веб-сайт: www.rock-chips.com

Телефон службы поддержки: +86-4007-700-590

Факс службы поддержки: +86-591-83951833

Электронная почта службы поддержки: fae@rock-chips.com

Предисловие

Rockit позиционируется как универсальная мультимедийная pipeline-платформа: часто используемые мультимедийные компоненты оформлены в виде плагинов, а гибкий pipeline приложения строится из них как из блоков. Разработчики могут использовать Rockit для создания разнообразных мультимедийных приложений.

Обзор

В настоящем документе приведены справочные сведения по мультимедийной разработке с Rockit. В основном рассматриваются интерфейсы разработки приложений Rockit, топология мультимедийного pipeline, типы и параметры мультимедийных плагинов, разработка пользовательских плагинов и другие вопросы.

Версии продуктов

Название микросхемы	Версия ядра
RV1126/RV1109	Linux 4.19

Целевая аудитория

Настоящий документ (руководство) предназначен главным образом для следующих специалистов:

инженеров технической поддержки;

инженеров-разработчиков программного обеспечения.

История изменений

Номер версии	Автор	Дата изменения	Описание изменения
V0.6.1	**Чэн Минчуань**	2020-09-11	**Возможности и интерфейсы Rockit**
V0.6.2	**Сюй Лимин**	2020-09-11	**Разработка плагинов и приложений Rockit**
V0.7.0	**Чэн Минчуань**	2020-09-15	**Усовершенствование интерфейсов разработки приложений Rockit**
V0.8.0	**Фан Синвэнь**	2020-10-28	**Настройка UVC-приложений и усовершенствование части интерфейсов**

Содержание

Руководство разработчика Rockchip Linux Rockit
- Возможности Rockit
- Интерфейсы разработки приложений Rockit
  - Интерфейс плагина (RTTaskNode)
  - Интерфейс графа задач (TaskGraph)
  - Операции управления RTTaskGraph
  - Интерфейс наблюдения за выходными данными TaskGraph
  - Порядок обработки изменений ключевого потока данных
    - Изменение разрешения
    - Изменение формата данных
  - Описание параметров основных плагинов
- Разработка плагина Rockit
  - Регистрация узла
  - Ключевые функции узла
  - Описание функций контекста
- Разработка приложения Rockit
  - Автоматическое построение приложения Rockit (рекомендуется)
    - Пример файла конфигурации графа
    - Список параметров конфигурации графа
    - Пример автоматически построенного приложения
  - Ручное построение приложения Rockit (не рекомендуется)
    - Пример ручного построения
    - Пример ручного создания плагина
    - Пример ручного связывания плагинов
- Расширение существующего приложения
  - Настройка UVC-приложения
    - Сокращение функциональности UVC
    - Переопределение UVC-плагина
    - Настройка связей узлов
  - Настройка UAC-приложения (добавлен плагин Audio 3A)

Возможности Rockit

Фреймворк Rockit обладает следующими возможностями:

- **стабильная абстракция операционных интерфейсов;**
- **стабильная абстракция мультимедийных интерфейсов, преобразующая платформенные мультимедийные интерфейсы в универсальные;**
- **стабильная абстракция плагинов;**
- **поддержка универсального управления плагинами (TaskGraph), включая сборку плагинов, передачу данных, управление и другие операции;**
- **поддержка разработки различных мультимедийных приложений.**

Мультимедийные плагины, поддерживаемые фреймворком Rockit в настоящее время:

Текущие стабильные плагины приведены в таблице ниже. Макроопределения имён плагинов приведены в RTNodeCommon.h.

Особое примечание: примеров настройки параметров плагинов очень много; см. конфигурации в SDK.

Имя файла: ${rv1126_sdk}/external/rockit/sdk/conf/aicamera_rockx.json

Имя файла: ${rv1126_sdk}/external/rockit/sdk/conf/aicamera_stasteria.json

Тип	Имя плагина	Назначение плагина	Тип входа	Тип выхода
Device	rkisp	RK-ISP	N/A	multi(NV12 etc)
Device	alsa_capture	RK-alsa	N/A	PCM
Device	alsa_playback	RK-alsa	PCM	N/A
Codec	rkmpp_dec	**Кодировщик RK-MPP**	NV12	H264/H265/MJPEG
Codec	rkmpp_enc	**Декодер RK-MPP**	H264/H265/MJPEG	NV12
Filter	alg_3a / alg_anr	RK-Audio3A-Old	PCM	PCM
Filter	skv_aec / skv_agc / skv_bf	RK-Audio3A-Skv	PCM	PCM
Filter	resample	Audio-Resample	PCM	PCM
Filter	rkrga	RK-RGA 2D **Ускорение**	multi	multi
Filter	rockx	RK-NPU AI **Ускорение**	**multi (предпочтительно NV12)**	RTResult
Filter	st_asteria	ST Asteria **Ускорение ИИ**	**multi (предпочтительно NV12)**	STResult
Filter	rkeptz	**Составной плагин EPTZ**	**multi (предпочтительно NV12)**	RTResult

Интерфейсы разработки приложений Rockit

Интерфейс плагина (RTTaskNode)

- Область применения: разработка плагинов инженерами-программистами Рекомендация: разрабатывать собственный плагин следует только при отсутствии требуемого плагина во фреймворке; по возможности не создавайте собственные плагины.
-
- Связанный файл: RTTaskNode.h

Базовым классом плагина является RTTaskNode. После упаковки функционального модуля (например ISP/RGA/MPP/Rockx) в плагин (RTTaskNode) универсальный TaskGraph может гибко вызывать эти плагины и строить различные сценарии приложения в соответствии с файлом конфигурации. При разработке плагина необходимо понимать следующие интерфейсы.

```
// Входной параметр: context — контекст узла для хранения различных параметров, необходимых при инициализации узла.
// Выход: RT_RET — RT_OK при успешном выполнении, другое значение при ошибке. После ошибки успешно создать узел невозможно.
// Функция: инициализация узла.
virtual RT_RET RTTaskNode::open(RTTaskNodeContext *context);

// Входной параметр: context — контекст узла для хранения различных параметров, входных и выходных данных, необходимых при обработке узла.
// Выход: RT_RET — RT_OK при успешном выполнении, другое значение при ошибке.
// Функция: обработка данных узла. RT_RET: RT_OK при успешном выполнении, другое значение при ошибке.
virtual RT_RET RTTaskNode::process(RTTaskNodeContext *context);

// Входной параметр: context — контекст узла для хранения различных параметров, необходимых при деинициализации узла. // Выход: RT_RET — RT_OK при успешном выполнении, другое значение при ошибке.
// Функция: деинициализация узла.
virtual RT_RET RTTaskNode::close(RTTaskNodeContext *context);

// Входной параметр: meta — параметры конфигурации пользователя.
// Выход: RT_RET — RT_OK при успешном выполнении, другое значение при ошибке.
// Функция: настройка параметров узла пользователем.
virtual RT_RET RTTaskNode::invoke(RtMetaData *meta);
```

К связанным с RTTaskNode классам относятся InputStreamManager, OutputStreamManager, InputStreamHandler, OutputStreamHandler и RTTaskNodeContext.

Интерфейс графа задач (TaskGraph)

- Область применения: разработка приложений инженерами-программистами Рекомендация: разрабатывайте приложение через интерфейс графа задач (TaskGraph), не управляйте плагинами и потоками данных самостоятельно через интерфейс уровня плагинов.
-
- Связанный файл: RTTaskGraph.h

Базовым классом графа задач является RTTaskGraph. Основные функции графа задач (RTTaskGraph):

- управление файлом конфигурации и инициализацией системы плагинов;
- управление потоком данных системы плагинов, включая ввод, вывод и передачу данных;
- управление потоком управления системы плагинов, включая init/prepare/start/stop и другие операции;
- управление выполнением задач системы плагинов.

Принцип работы плагина следующий: Scheduler извлекает атомарную задачу из TaskNode и передаёт её в SchedulerQueue, а исполнитель (Executor) извлекает атомарную задачу из SchedulerQueue и направляет её конкретному исполнителю (например ThreadPool). Выполнение задачи плагина можно рассматривать как «чёрный ящик»: следить за внутренним процессом не требуется, достаточно учитывать вход и выход RTTaskGraph.

Граф задач (RTTaskGraph) является классом для разработки приложений. При разработке приложения необходимо понимать следующие интерфейсы.

```
// Входной параметр: configFile — файл конфигурации JSON.
// Выход: RT_RET — RT_OK при успешном выполнении, другое значение при ошибке.
// Функция: создание TaskGraph в соответствии с пользовательской конфигурацией.
RT_RET   RTTaskGraph::autoBuild(const char* configFile);

// Входной параметр: параметры, переопределяющие настройки файла конфигурации.
// Выход: RT_RET — RT_OK при успешном выполнении, другое значение при ошибке.
```

```
// Функция: подготовка внутренних ресурсов выполнения.
RT_RET    RTTaskGraph::prepare(RtMetaData *params = RT_NULL);

// Входной параметр: NONE.
// Выход: RT_RET — RT_OK при успешном выполнении, другое значение при ошибке.
// Функция: запуск графа задач.
RT_RET    RTTaskGraph::start();

// Входной параметр: по умолчанию блокирующий режим с ожиданием остановки. Неблокирующий режим обычно применяется для асинхронных операций совместно с waitUntilDone. // Выход: RT_RET — RT_OK при успешном выполнении, другое значение при ошибке.
// Функция: остановка графа задач.
RT_RET    RTTaskGraph::stop(RT_BOOL block = RT_TRUE);

// Входной параметр: NONE.
// Выход: RT_RET — RT_OK при успешном выполнении, другое значение при ошибке.
// Функция: возобновление работы графа задач.
RT_RET    RTTaskGraph::resume();

// Входной параметр: NONE.
// Выход: RT_RET — RT_OK при успешном выполнении, другое значение при ошибке.
// Функция: приостановка работы графа задач.
RT_RET    RTTaskGraph::pause();

// Входной параметр: NONE.
// Выход: RT_RET — RT_OK при успешном выполнении, другое значение при ошибке.
// Функция: очистка внутренних данных графа задач.
RT_RET    RTTaskGraph::flush();

// Входной параметр: NONE.
// Выход: RT_RET — RT_OK при успешном выполнении, другое значение при ошибке.
// Функция: освобождение внутренних ресурсов графа задач при завершении.
RT_RET    RTTaskGraph::release();

// Входные параметры: cmd — см. определение перечисления; params — дополнительные параметры cmd.
// Выход: RT_RET — RT_OK при успешном выполнении, другое значение при ошибке.
// Функция: пользовательский расширяемый API для настройки параметров узла согласно пользовательской конфигурации.
RT_RET    RTTaskGraph::invoke(INT32 cmd, RtMetaData *params);

// Входные параметры: streamName — имя потока; streamId — идентификатор потока; streamCallback — callback потока.
// Выход: NONE.
// Функция: наблюдение за указанным выходным потоком.
RT_RET    RTTaskGraph::observeOutputStream(const std::string& streamName,
              INT32 streamId,
              std::function<RT_RET(RTMediaBuffer *)> streamCallback);

// Входной параметр: streamId — идентификатор потока.
// Выход: RT_RET — RT_OK при успешном выполнении, другое значение при ошибке.
// Функция: отмена наблюдения за указанным выходным потоком.
RT_RET    RTTaskGraph::cancelObserveOutputStream(INT32 streamId);

// Входной параметр: NONE.
// Выход: RT_RET — RT_OK при успешном выполнении, другое значение при ошибке.
// Функция: ожидание вывода наблюдателя. Интерфейс требуется вызывать только при организации синхронного вывода.
RT_RET    RTTaskGraph::waitForObservedOutput();

// Входной параметр: NONE.
// Выход: RT_RET — RT_OK при успешном выполнении, другое значение при ошибке.
// Функция: ожидание завершения работы; главным образом применяется для блокирования функции MAIN, чтобы она не завершалась.
```

```
RT_RET   RTTaskGraph::waitUntilDone();
```

К связанным с RTTaskGraph классам относятся TaskNode, Scheduler, SchedulerQueue, Executor и ThreadPool. Эти классы являются внутренними и не требуют внимания при разработке приложения Rockit.

Операции управления RTTaskGraph

После чтения файла конфигурации плагинов RTTaskGraph автоматически строит Pipeline и начинает работу. Посредством небольшого числа интерфейсов можно обеспечить согласованную обработку сложного потока данных в Pipeline. Для разработки сложного приложения разработчику достаточно учитывать конфигурацию плагинов и RTTaskGraph.

```
RTTaskGraph *graph = new RTTaskGraph();
graph->autoBuild("your_graph.json");
// Подготовка pipeline, включая подготовку плагинов, соединение входов и выходов и другие операции.
graph->prepare();
// Запуск pipeline и открытие всего потока данных.
graph->start();
// Остановка pipeline.
graph->stop();

// Настройка параметров pipeline.
// Настройка параметров фреймворка: GRAPH_CMD_PRIVATE_CMD.
// Настройка параметров узла: GRAPH_CMD_TASK_NODE_PRIVATE_CMD.
RtMetaData params;
params.setInt32(kKeyTaskNodeId, nodeId);
params.setCString(kKeyPipeInvokeCmd, "qp-control");
params.setInt32("qp_init",  24);
params.setInt32("qp_step",  8);
params.setInt32("qp_min",   4);
params.setInt32("qp_max",   24);
params.setInt32("min_i_qp", 4);
params.setInt32("max_i_qp", 24);
graph->invoke(GRAPH_CMD_TASK_NODE_PRIVATE_CMD, &params);
```

Интерфейс наблюдения за выходными данными TaskGraph

Разработчик приложения должен учитывать вход и выход RTTaskGraph. Плагины входа/источника данных RTTaskGraph (NodeIsp/NodeDemuxer) являются самоуправляемыми входами (данные потребляет нижестоящий уровень), поэтому достаточно учитывать настройку таких плагинов.

Sink-плагин RTTaskGraph является самоуправляемым выходом (данные потребляет текущий уровень), поэтому достаточно учитывать настройку такого плагина. Filter-плагин RTTaskGraph (AIVisionFilter) обычно выполняет промежуточную обработку; при использовании такого плагина как выхода разработчик приложения должен получить его данные (AIVisionData) для дополнительной постобработки.

```
// Пользовательская функция постобработки OBSERVER, обычно применяемая для сопряжения выходных данных приложения и ROCKIT.
RT_RET YOUR_OBSERVER_FUNC(RTMediaBuffer *buffer) {
// Следующий пример: получение результата обнаружения ИИ.
    RTAIDetectResults *aiResult  = RT_NULL;
    buffer->getMetaData()->findPointer(OPT_AI_DETECT_RESULT,
                          reinterpret_cast<RT_PTR *>(&aiResult));
    if (!strcmp(aiResult->vendor, "rockx")) {
        RTRknnAnalysisResults *result = (RTRknnAnalysisResults *)aiResult-
>privData;
    } else {
        OtherAnalysisResults *result = (OtherAnalysisResults *)aiResult-
>privData;
    }
```

```
    buffer->release();
    return RT_OK;
}

RTTaskGraph *graph = new RTTaskGraph();
graph->autoBuild("your_ai_vision.json");
// Подготовка pipeline, включая подготовку плагинов, соединение входов и выходов и другие операции.
graph->preare();
// Наблюдение за выходом pipeline.
graph->observeOutputStream("ai_rockx", ${stream_id} << 16, YOUR_OBSERVER_FUNC);
// Запуск pipeline и открытие всего потока данных.
graph->start();
// Другие операции...
```

Порядок обработки изменений ключевого потока данных

Изменение разрешения

@TODO

Изменение формата данных

@TODO

Описание параметров основных плагинов

Особое примечание: примеров настройки параметров плагинов очень много; см. конфигурации в SDK.

Имя файла: ${rv1126_sdk}/external/rockit/sdk/conf/aicamera_rockx.json

Имя файла: ${rv1126_sdk}/external/rockit/sdk/conf/aicamera_stasteria.json

Плагин RKISP

См. указанный выше файл конфигурации в SDK.

Плагин RKRGA

См. указанный выше файл конфигурации в SDK.

Плагин RKROCKX

См. указанный выше файл конфигурации в SDK.

Плагин RKMPP

См. указанный выше файл конфигурации в SDK.

Плагины, связанные с UAC

См. документ «Руководство разработчика приложения Rockchip Linux UAC» в Linux SDK.

Разработка плагина Rockit

Ниже структура конфигурации графа рассматривается на примере demo.

```
// Создание внешнего узла; внешний узел должен наследовать RTTaskNode.
// Необходимо реализовать базовые интерфейсы open/process/close.
class RTRockitDemoNode : public RTTaskNode {
```

```
 public:
    RTRockitDemoNode() {}
    virtual ~RTRockitDemoNode() {}

    virtual RT_RET open(RTTaskNodeContext *context) { return RT_OK; }
    virtual RT_RET process(RTTaskNodeContext *context);
    virtual RT_RET close(RTTaskNodeContext *context) { return RT_OK; }
};

// Используется для создания узла; указатель на функцию сохраняется в RTNodeStub.mCreateObj.
static RTTaskNode* createRockitDemoNode() {
    return new RTRockitDemoNode();
}

// Функция обработки узла обрабатывает входные данные и затем выводит их нижестоящему узлу.
RT_RET RTRockitDemoNode::process(RTTaskNodeContext *context) {
    RTMediaBuffer *inputBuffer  = RT_NULL;
    RTMediaBuffer *outputBuffer = RT_NULL;

// Проверка пустоты входа.
    if (context->inputIsEmpty()) {
        return RT_ERR_BAD;
    }

// Извлечение входного buffer.
    inputBuffer = context->dequeInputBuffer();
// Получение свободного выходного Buffer.
    outputBuffer = context->dequeOutpuBuffer(RT_TRUE, inputBuffer->getLength());
    if (RT_NULL == outputBuffer) {
        inputBuffer->release();
        return RT_ERR_BAD;
    }
// Копирование входных данных в выход (в demo выполняется простейшее копирование).
    rt_memcpy(outputBuffer->getData(), inputBuffer->getData(), inputBuffer-
>getLength());
// Настройка диапазона выходного buffer.
    outputBuffer->setRange(0, inputBuffer->getLength());
// Установка признака EOS.
    if (inputBuffer->isEOS()) {
        outputBuffer->getMetaData()->setInt32(kKeyFrameEOS, 1);
    }
// После завершения использования входного Buffer вызывается освобождение.
    inputBuffer->release();
// Передача выходного buffer наружу и завершение обработки.
    context->queueOutputBuffer(outputBuffer);

    return RT_OK;
}

// Заглушка сведений об узле, используемая для регистрации узла.
RTNodeStub node_stub_rockit_demo {
// UID узла — уникальный идентификатор узла (0–1000).
    .mUid           = kStubRockitDemo,
// Имя узла, главным образом используемое для поиска и создания узла.
// corp_role_name — имя должно быть уникальным.
    .mName          = "rockit_demo",
// Номер версии.
    .mVersion       = "v1.0",
```

```
// Метод создания узла; преобразован в макроопределение.
    .mCreateObj    = createRockitDemoNode,
    .mCapsSrc      = { "video/x-raw", RT_PAD_SRC,  {RT_NULL, RT_NULL} },
    .mCapsSink     = { "video/x-raw", RT_PAD_SINK, {RT_NULL, RT_NULL} },
};

// Проверка повторения UUID и сообщение об ошибке.
RT_NODE_FACTORY_REGISTER_STUB(node_stub_rockit_demo);
```

Регистрация узла

Для регистрации узла требуется заглушка; основные сведения заглушки приведены ниже.

```
// Заглушка сведений об узле, используемая для регистрации узла.
RTNodeStub node_stub_rockit_demo {
// UID узла — уникальный идентификатор узла.
    .mUid          = kStubRockitDemo,
// Имя узла, главным образом используемое для поиска и создания узла.
    .mName         = "rockit_demo",
// Номер версии.
    .mVersion      = "v1.0",
// Метод создания узла.
    .mCreateObj    = createRockitDemoNode,
    .mCapsSrc      = { "video/x-raw", RT_PAD_SRC,  {RT_NULL, RT_NULL} },
    .mCapsSink     = { "video/x-raw", RT_PAD_SINK, {RT_NULL, RT_NULL} },
};
```

В месте определения заглушки глобально вызовите следующую функцию для завершения регистрации.

```
RT_NODE_FACTORY_REGISTER_STUB(node_stub_rockit_demo);
```

Ключевые функции узла

```
// Входной параметр: context — контекст узла для хранения различных параметров, необходимых при инициализации узла.
// Выход: RT_RET — RT_OK при успешном выполнении, другое значение при ошибке. После ошибки успешно создать узел невозможно.
// Функция: инициализация узла.
RT_RET open(RTTaskNodeContext *context);

// Входной параметр: context — контекст узла для хранения различных параметров, входных и выходных данных, необходимых при обработке узла.
// Выход: RT_RET — RT_OK при успешном выполнении, другое значение при ошибке.
// Функция: обработка данных узла. RT_RET: RT_OK при успешном выполнении, другое значение при ошибке.
RT_RET process(RTTaskNodeContext *context);

// Входной параметр: context — контекст узла для хранения различных параметров, необходимых при деинициализации узла. // Выход: RT_RET — RT_OK при успешном выполнении, другое значение при ошибке.
// Функция: деинициализация узла.
RT_RET close(RTTaskNodeContext *context);
```

Описание функций контекста

RTTaskNodeContext хранит сведения, необходимые для инициализации, обработки и деинициализации узла.

```
// Хранение параметров узла. Например, для узла кодирования здесь могут храниться width, height, bitrate и другие сведения.
```

```
RtMetaData* options();

// Входной параметр: streamType — тип входного пакета. По умолчанию none; при нескольких входах определяет очередь входных пакетов, из которой извлекаются данные. // Выход: RTMediaBuffer — входной пакет RTMediaBuffer.
// Функция: получение входного пакета.
RTMediaBuffer *dequeInputBuffer(std::string streamType = "none");

// Входные параметры:
//   block — признак блокирования. По умолчанию блокирующий режим: ожидается свободный packet, а при его наличии свободный выходной пакет немедленно возвращается.
//   size — требуемый размер packet; будет возвращён buffer размером не меньше size.
//   streamType — тип выходного пакета. По умолчанию none; при нескольких выходах определяет выходной пул, из которого берётся свободный пакет.
buffer
// Выход: RTMediaBuffer — свободный выходной пакет RTMediaBuffer.
// Функция: получение свободного выходного пакета.
RTMediaBuffer *dequeOutputBuffer(
                        RT_BOOL block = RT_TRUE,
                        UINT32 size = 0,
                        std::string streamType = "none");

// Входные параметры:
//   packet — выходные данные, созданные process.
//   streamType — тип выходного пакета. По умолчанию none; при нескольких выходах определяет вход нижестоящего узла, которому направляется поток.
// Выход: RT_RET — RT_OK при успехе, другое значение при ошибке.
// Функция: сохранение выходного пакета и направление его нижестоящему узлу.
RT_RET queueOutputBuffer(RTMediaBuffer *packet, std::string streamType =
"none");

// Входной параметр: streamType — тип входа. По умолчанию none; при нескольких входах определяет очередь входа, пустота которой проверяется. // Выход: RT_BOOL — RT_TRUE, если пусто; RT_FALSE, если не пусто.
// Функция: проверка пустоты входа.
RT_BOOL inputIsEmpty(std::string streamType = "none");

// Входной параметр: streamType — тип выхода. По умолчанию none; при нескольких выходах определяет очередь выхода, пустота которой проверяется. // Выход: RT_BOOL — RT_TRUE, если пусто; RT_FALSE, если не пусто.
// Функция: проверка пустоты выхода.
RT_BOOL outputIsEmpty(std::string streamType = "none");
```

Разработка приложения Rockit

В настоящее время поддерживается автоматическое и ручное построение приложений на основе фреймворка ROCKIT. Автоматическое построение приложения Rockit означает использование файла конфигурации JSON для автоматического создания плагинов и PIPELINE. Ручное построение означает, что разработчик вручную создаёт плагины и PIPELINE. При автоматическом построении приложения достаточно знать элементы конфигурации и небольшое число интерфейсов, поэтому рекомендуется использовать этот способ. Основные этапы разработки приложения на основе фреймворка ROCKIT:

- оценить, удовлетворяют ли существующие плагины ROCKIT требованиям приложения и требуется ли разработка или расширение нового плагина;
- определить параметры плагинов, связи и отношения управления, затем настроить плагины TaskGraph согласно требованиям приложения;
- загрузить в приложении файл конфигурации JSON и выполнить сопряжение потоков данных и управления приложения с ROCKIT;
- **проверить бизнес-логику и стабильность приложения.**

Demo приложения находится в ${SDK_ROOT}/external/rockit.

Автоматическое построение приложения Rockit (рекомендуется)

В настоящее время поддерживается автоматическое и ручное построение приложений на основе фреймворка ROCKIT. Автоматическое построение приложения Rockit означает использование файла конфигурации JSON Файл конфигурации автоматически создаёт плагины и PIPELINE. Интерфейс автоматического построения приложения Rockit достаточно удобен; инженерам приложений рекомендуется использовать этот способ разработки приложений на основе фреймворка ROCKIT.

Пример файла конфигурации графа

```
{
// Первый уровень конфигурации: pipe_0 представляет один граф. В настоящее время в одной конфигурации нельзя задавать несколько графов, поэтому обычно здесь // используется "pipe_0".
    "pipe_0": {
// Второй уровень конфигурации: сведения о настройке узла.
        "node_0": {
// Общие сведения об узле; в настоящее время только имя узла.
            "node_opts": {
                "node_name"       : "rkisp"
            },
// Сведения context узла, например источник данных, тип, количество и размер выходных buffer.
            "node_opts_extra": {
                "node_source_uri" : "/dev/media1",
                "node_buff_type"  : 0,
                "node_buff_count" : 4,
                "node_buff_size"  : 460800
            },
// Общие сведения о потоке данных, например тип и имя.
            "stream_opts": {
                "stream_output"   : "image:nv12_0",
                "stream_fmt_out"  : "image:nv12"
            },
// Индивидуальные сведения узла; в данном случае они будут настроены внутри узла rkisp.
            "stream_opts_extra": {
                "opt_entity_name" : "rkispp_scale1",
                "opt_width"       : 640,
                "opt_height"      : 480,
                "opt_vir_width"   : 640,
                "opt_vir_height"  : 480,
                "opt_buf_type"    : 1,
                "opt_mem_type"    : 4,
                "opt_use_libv4l2" : 1,
                "opt_colorspace"  : 0
            }
        },
        "node_1": {
            "node_opts": {
                "node_name"       : "rkmpp_enc"
            },
            "node_opts_extra": {
                "node_buff_type"  : 0,
                "node_buff_count" : 4,
                "node_buff_size"  : 460800
            },
            "stream_opts": {
                "stream_input"    : "image:nv12_0",
                "stream_output"   : "image:h264_0",
                "stream_fmt_in"   : "image:nv12",
                "stream_fmt_out"  : "image:h264"
            },
            "stream_opts_extra": {
                "opt_width"      : 640,
                "opt_height"     : 480,
                "opt_vir_width"  : 680,
                "opt_vir_height" : 480,
```

```
                "opt_bitrate"    : 1000000,
                "opt_codec_type" : 6,
                "opt_frame_rate" : 30,
                "opt_profile"    : 100,
                "opt_level"      : 52,
                "opt_gop"        : 30,
                "opt_qp_init"    : 24,
                "opt_qp_step"    : 4,
                "opt_qp_min"     : 12,
                "opt_qp_max"     : 48
            }
        },
        "node_2": {
            "node_opts": {
                "node_name"       : "fwrite"
            },
            "node_opts_extra": {
                "node_source_uri" : "/data/test.yuv",
                "node_buff_type"  : 1,
                "node_buff_count" : 0
            },
            "stream_opts": {
                "stream_input"    : "image:h264_0",
                "stream_fmt_in"   : "image:h264"
            }
        }
}
```

Список параметров конфигурации графа

Приведены только общие сведения. Необщие сведения графа и узла следует запрашивать внутри кода соответствующего узла.

Приведённые параметры конфигурации относятся только к текущей версии; будущие версии могут содержать и другие параметры.

Имена макроопределений соответствуют макроопределениям в RTNodeCommon.h.

Имя параметра	Имя макроопределения	Функция	Является обязательной	Примечание информацией
node_name	OPT_NODE_NAME	Имя узла	Да	Система находит соответствующий узел по имени и завершает построение.
node_source_uri	OPT_NODE_SOURCE_URI	Источник данных	Нет	Узел использует это значение для открытия файла, устройства и других объектов ввода-вывода.
node_buff_type	OPT_NODE_BUFFER_TYPE	Тип выхода buffer	Нет	Значение 0 означает, что buffer выделяется внутри узла; значение 1 означает, что buffer выделяется вне узла, а узел предоставляет только структуру RTMediaBuffer.
node_buff_count	OPT_NODE_BUFFER_COUNT	Тип выхода buffer Количество	Да	Значение 0 означает, что выделение не выполняется.
node_buff_size	OPT_NODE_BUFFER_SIZE	Тип выхода buffer Размер	Нет	
stream_input	OPT_STREAM_INPUT_NAME	Имя входного потока данных	Нет	Используется для связывания потока данных узла; должно соответствовать stream_output вышестоящего узла. Если внутри узла имеется несколько входов, для их различения используйте stream_input_ + index, например stream_input_0。
stream_output	OPT_STREAM_OUTPUT_NAME	Имя входного потока данных Имя выхода	Нет	Используется для связывания потока данных узла; должно Соответствует stream_input нижестоящего узла. Если внутри узла имеется несколько выходов, используйте stream_output_ + index. stream_output_0。
stream_fmt_in	OPT_STREAM_FMT_IN	Имя входного потока данных Тип входа	Нет	Определяет тип входных данных; внутри узла этот тип используется для определения типа потока данных. При нескольких входах он должен соответствовать stream_input, например stream_fmt_in_0。
stream_fmt_out	OPT_STREAM_FMT_OUT	Имя входного потока данных Тип выхода	Нет	Определяет тип выходных данных; этот тип используется для определения типа потока данных. При нескольких входах он должен соответствовать должен соответствовать stream_output, например stream_fmt_out_0。

Пример автоматически построенного приложения

Ниже описано, как использовать приведённый файл конфигурации для автоматического построения графа.

```
#define RT_GRAPH_TRANSCODING_FILE
"/data/file_h264_rkmpp_dec_rkmpp_h265_enc_write.json"

RT_RET unit_test_graph_transcoding(INT32 index, INT32 total) {
    (void)index;
    (void)total;

// Создание пустого графа.
    RTTaskGraph *transcodingGraph = new RTTaskGraph("UVCGraphTranscodingTest");
// Автоматическое построение по файлу конфигурации.
    transcodingGraph->autoBuild(RT_GRAPH_TRANSCODING_FILE);
// Подготовка графа.
    transcodingGraph->perpare();
// Запуск графа.
    transcodingGraph->start();
    // ....
// Остановка графа.
    transcodingGraph->stop();
    transcodingGraph->release();
// Уничтожение графа.
    rt_safe_delete(transcodingGraph);

    return RT_OK;
}
```

Ручное построение приложения Rockit (не рекомендуется)

В настоящее время поддерживается автоматическое и ручное построение приложений на основе фреймворка ROCKIT. Ручное построение означает, что разработчик вручную создаёт плагины и PIPELINE. При автоматическом построении приложения достаточно знать элементы конфигурации и небольшое число интерфейсов; рекомендуется использовать автоматический способ.

Пример ручного построения

```
// Создание пустого графа.
RTTaskGraph *demoGraph = new RTTaskGraph("rockit_demo");
// Создание узла по сведениям конфигурации.
RT_NODE_CONFIG_STRING_APPEND(nodeConfig, XXX, XXX);
RT_NODE_CONFIG_STRING_APPEND(streamConfig, XXX, XXX);
RTTaskNode *freadNode = demoGraph->createNode(nodeConfig, streamConfig);
XXX
......
// Связать поток данных узлов
demoGraph->linkNode(freadNode, demoNode, fwriteNode);
// Подготовка ресурсов графа
demoGraph->prepare();
// Запуск работы графа
demoGraph->start();
```

Ниже приведены подробные этапы.

Пример ручного создания плагина

Следующий пример показывает ручное создание узла. Порядок использования параметров узла см. в разделе «4.2 Автоматическое построение приложения Rockit».

```
// Определение конфигурации узлов и конфигурации потоков данных
```

```
std::string nodeConfig;
std::string streamConfig;
// RT_NODE_CONFIG_STRING_APPEND добавляет сведения о конфигурации
RT_NODE_CONFIG_STRING_APPEND      (nodeConfig, OPT_NODE_NAME,
 NODE_NAME_FREAD);
RT_NODE_CONFIG_STRING_APPEND      (nodeConfig, OPT_NODE_SOURCE_URI,
 "/data/test.h264");
RT_NODE_CONFIG_NUMBER_APPEND      (nodeConfig, OPT_NODE_BUFFER_TYPE,      0);
RT_NODE_CONFIG_NUMBER_APPEND      (nodeConfig, OPT_NODE_BUFFER_COUNT,     4);
RT_NODE_CONFIG_NUMBER_APPEND      (nodeConfig, OPT_NODE_BUFFER_SIZE,      1024 *
1024);
// RT_NODE_CONFIG_NUMBER_LAST_APPEND добавляет последний элемент конфигурации
RT_NODE_CONFIG_NUMBER_LAST_APPEND(nodeConfig, OPT_FILE_READ_SIZE,       1024 *
1024);
RT_NODE_CONFIG_STRING_APPEND      (streamConfig, OPT_STREAM_OUTPUT_NAME,
"fread_out");
RT_NODE_CONFIG_STRING_LAST_APPEND(streamConfig, OPT_STREAM_FMT_OUT,
"image:h264");
// Создание узла
RTTaskNode *freadNode = demoGraph->createNode(nodeConfig, streamConfig);
```

В приведённом выше способе создания OPT_NODE_NAME — имя узла, используемое для поиска соответствующего узла в node factory; оно обязательно должно быть указано верно, иначе создание завершить невозможно.

Пример ручного связывания плагинов

Связывание узлов:

```
// Входные параметры:
//   srcNode: вышестоящий связываемый узел, предоставляющий выход связи.
//   dstNode: нижестоящий связываемый узел, предоставляющий вход связи.
//      ...: поддерживается передача нескольких узлов // Выходной параметр: RT_RET; при успешном связывании возвращается RT_OK, при ошибке — значение, указывающее конкретную причину.
// Назначение функции: связывает узлы. После связывания выходные данные, сформированные process вышестоящего узла, поступают во входную очередь // нижестоящего узла.
//     Следует учитывать, что для связывания должны совпадать типы входов и выходов узлов и прочие сведения.
RT_RET linkNode(RTTaskNode *srcNode, RTTaskNode *dstNode, ...);
```

Отмена связывания узлов:

```
// Входные параметры:
//   srcNode: вышестоящий связываемый узел, предоставляющий выход связи.
//   dstNode: нижестоящий связываемый узел, предоставляющий вход связи.
// Выходной параметр: RT_RET; при успешном связывании возвращается RT_OK, при ошибке — значение, указывающее конкретную причину.
// Назначение функции: отменяет связь между плагинами.
RT_RET   unlinkNode(RTTaskNode *srcNode, RTTaskNode *dstNode);
```

Расширение существующего приложения

В настоящее время встроена поддержка приложений UVC+NN и UAC. Интерфейсам приложений UVC и NN соответствуют RTUVCGraph и RTUACGraph; подробности см. в соответствующих заголовочных файлах в external\\rockit\\sdk\\headers. Ниже описано, как расширять существующие приложения и переопределять отдельные плагины.

Настройка UVC-приложения

Сокращение функциональности UVC

Поддерживаемые приложением RTUVCGraph функции ZOOM, Pan&Tile, EPTZ и NN можно исключать в соответствии с требованиями проекта, сокращая потребление памяти и других ресурсов. В external\\rockit\\sdk\\conf представлены следующие варианты сокращённой конфигурации, которые можно использовать как основу для индивидуальной настройки.

Имя файла конфигурации	Описание функции
aicamera_uvc.json	**Только функция предварительного просмотра UVC**
aicamera_rockx_only.json	**Только функция Rockx**
aicamera_uvc_zoom.json	**Поддержка UVC, ZOOM и Pan&Tile**
aicamera_uvc_zoom_eptz.json	**Поддержка UVC, ZOOM, Pan&Tile и EPTZ**
aicamera_uvc_zoom_rockx.json	**Поддержка UVC, ZOOM, Pan&Tile и Rockx**
aicamera_uvc_zoom_eptz_rockx.json	**Поддержка UVC, ZOOM, Pan&Tile, EPTZ и Rockx**

Переименуйте выбранный выше файл конфигурации в aicamera_rockx.json, чтобы заменить исходный набор функций сокращённым.

Переопределение UVC-плагина

Если встроенные плагины не удовлетворяют требованиям проекта, можно создать собственный плагин и заменить им плагин в pipeline UVC. Порядок действий:

- **Разработка собственного плагина Создайте собственный плагин в каталоге проекта app\\aiserver\\src\\vendor. В этом каталоге имеются примеры плагинов eptz, rockx и др., которые можно использовать как образец. Реализацию нового узла см. в разделе «Как разрабатывать плагины rockit».**
- **Изменение имени плагина в файле конфигурации Измените external\\rockit\\sdk\\conf\\aicamera_rockx.json, заменив имя требуемого плагина именем собственного плагина.**

После выполнения этих двух шагов пересоберите образ прошивки, чтобы завершить настройку плагина UVC. Если требуется изменить связи между узлами,

см. приведённое ниже описание «Настройка связей узлов».

Настройка связей узлов

Конфигурацию узлов в файле конфигурации см. в разделе «Пример файла конфигурации графа». Связи между узлами поддерживают как автоматическое связывание из файла конфигурации (путём сопоставления stream_output вышестоящего узла со stream_input нижестоящего узла), так и прямое задание вышестоящих и нижестоящих связей через NodeID (может применяться в сложных схемах связывания узлов). Ниже приведены примеры явного задания связей.

Пример 1: одна связь

```
"link_0": {
    "link_name"          : "uvc",
    "link_ship"          : "0,7"
},
```

link_name: функция данного сценария — UVC. Код включает UVC и ищет соответствующую связь по этому имени, поэтому имя изменять нельзя.

link_ship: "0,7" означает создание связи между узлами 0 и 7, то есть поток данных идёт от узла 0 к узлу 7.

Пример 2: несколько связей

```
"link_1": {
    "link_name"          : "nn_isp",
    "link_ship"          : "2,8-8,11,1000-8,12,1000-8,13,1000"
},
"link_2": {
    "link_name"          : "nn_linkout",
    "link_ship"          : "10,8-8,11,1000-8,12,1000-8,13,1000"
}
```

link_name: функция данного сценария — алгоритм NN. "nn_isp" означает, что источник данных алгоритма NN поступает от isp (узел 2 — ispp scale1); это имя также изменять нельзя. "nn_linkout" — тоже функция алгоритма NN, но отличается от "nn_isp" тем, что данные поступают от узла 10 (изображение после обрезки, масштабирования и другой обработки EPTZ).

link_ship: "2,8-8,11,1000-8,12,1000-8,13,1000". Символ «-» разделяет группы связей, а «,» — узлы внутри одной группы связей. Таким образом, "2,8" означает связь узла 2 с узлом 8; "8,11,1000" — связь узла 8 с узлом 11 и узла 11 с узлом 1000; "8,12,1000" — связь узла 8 с узлом 12 и узла 12 с узлом 1000. Узлы 11, 12 и 13 представляют соответственно алгоритмы распознавания лица, ключевых точек лица и скелета. Если все алгоритмы NN реализуются одним узлом, узлы NN можно объединить. Например, приведённую ниже конфигурацию можно заменить алгоритмом stasteria, при этом узел 11 также необходимо настроить как узел stasteria.

```
"link_1": {
    "link_name"          : "nn_isp",
    "link_ship"          : "2,8,11,1000"
},
"link_2": {
    "link_name"          : "nn_linkout",
    "link_ship"          : "10,8,11,1000"
}
```

Настройка UAC-приложения (добавлен плагин Audio 3A)

- Определение требований к функции UAC
- Оцените, какие мультимедийные плагины можно использовать повторно, а какие требуется разработать.
- По плагину обработки аудио 3A для UAC см. соответствующую документацию Rockchip Linux UAC.
- Определите топологию плагинов pipeline UAC и файл конфигурации.
- Добавьте плагин Audio 3A, реализовав его способом, описанным для собственных мультимедийных плагинов.